

```
%% Simulation Frequenzgang Drohen

clc, clear variables
addpath ../lib/
addpath ../20250907/
s = tf('s');

%% Laden von Drohnenflüge als Referenz
flight_folder = '20250907';

quad = 'apex5';
log_name = '20250907_apex5_00.bbl.csv';

% Choose an axis: 1: roll, 2: pitch, 3: yaw
ind_ax = 1;

%% Daten Laden,

file_path = fullfile(flight_folder, log_name);
[para, Nheader, ind, ind_cntr] = extract_header_information(file_path);

% Read the data
% - If its the first time from the .csv and save a mat, otherwise the
%   .mat. This increases load speed significantly.
tic
try
    load([file_path(1:end-8), '.mat'])
catch exception
    data = readmatrix(file_path, 'NumHeaderLines', Nheader);
    save([file_path(1:end-8), '.mat'], 'data');
end
[Ndata, Nsig] = size(data);
toc
% Expand index
ind.axisSumPI = ind_cntr + (1:3);
ind.sinarg = ind.debug(1);

%% Einstellungen Bodeplot

% Defines
set(cstprefs.tbxprefs, 'MagnitudeUnits', 'dB');
set(cstprefs.tbxprefs, 'FrequencyUnits', 'Hz');
set(cstprefs.tbxprefs, 'UnwrapPhase', 'Off');
set(cstprefs.tbxprefs, 'Grid', 'On');
set(groot, 'defaultLineLineWidth', 1.2); % global für alle neuen Linien
set(0, 'defaultAxesColorOrder', get_my_colors);

% Bodeoptions
opt = bodeoptions('cstprefs');
opt.Xlim = { [0.5 1e3] }; % x-Achse: 0.1 Hz bis 1000 Hz

%% Filterparameter
```

%Samplettime

```
Ts = para.looptime * 1.0e-6;           % Gyro loop
z = tf('z', Ts);
```

%Drohnenparameter

```
% type: 0: PT1, 1: BIQUAD, 2: PT2, 3: PT3
```

```
para_new.gyro_lpf           = 0;           % dono what this is
para_new.gyro_lowpass_hz    = 0;           % frequency of gyro lpf 1
para_new.gyro_soft_type     = 0;           % type of gyro lpf 1
para_new.gyro_lowpass_dyn_hz = [0, 0];     % dyn gyro lpf overwrites gyro_lowpass_hz
para_new.gyro_lowpass2_hz   = 800;        % frequency of gyro lpf 2
para_new.gyro_soft2_type    = 0;           % type of gyro lpf 2
para_new.gyro_notch_hz      = [0, 520];   % frequency of gyro notch 1 and 2
para_new.gyro_notch_cutoff  = get_fcut_from_D_and_fcenter([0.00, 0.15], para_new. ✓
gyro_notch_hz); % damping of gyro notch 1 and 2
para_new.dterm_lpf_hz       = 0;           % frequency of dterm lpf 1
para_new.dterm_filter_type  = 0;           % type of dterm lpf 1
para_new.dterm_lpf_dyn_hz   = [0, 0];     % dyn dterm lpf overwrites dterm_lpf_hz
para_new.dterm_lpf2_hz      = 130;        % frequency of dterm lpf 2
para_new.dterm_filter2_type = 3;           % type of dterm lpf 2
para_new.dterm_notch_hz     = 235;        % frequency of dterm notch
para_new.dterm_notch_cutoff = get_fcut_from_D_and_fcenter(0.15, para_new. ✓
dterm_notch_hz); % damping of dterm notch
para_new.yaw_lpf_hz         = 200;        % frequency of yaw lpf (pt1)
```

```
switch ind_ax
```

```
case 1 % roll: [49, 83, 33, 0]
    P_new      = 1.0 * 49;
    I_ratio_new = 1.0 * 83/83;
    D_new      = 1.0 * 33;
case 2 % pitch: [61, 103, 39, 0]
    P_new      = 1.0 * 61;
    I_ratio_new = 1.0 * 103/103;
    D_new      = 1.0 * 39;
case 3 % yaw: [42, 104, 3, 0]
    P_new      = 1.0 * 42;
    I_ratio_new = 1.0 * 104/104;
    D_new      = 1.0 * 3;
```

```
end
```

% Funktion für Tiefpassfilter

```
function G = Tiefpassfilter(type, omega, s)
```

```
if omega == 0
```

```
    G = tf(1);
```

```
else
```

```
    switch type
```

```
        case 0 % PT1
```

```
            G = 1 / (1 + s/omega);
```

```
        case 1 % BIQUAD
```

```
            G = 1 / (1 + s/omega); %unklar was für eine Übertragungsfunktion ✓
```

```
BIQUAD ist
```

```
        case 2 % PT2
```

```

        c = 1.553773974;
        G = 1 / (1 + s/(omega*c))^2;
    case 3 % PT3
        c = 1.961459177;
        G = 1 / (1 + s/(omega*c))^3;
    otherwise
        error('Unbekannter Filtertyp');

```

```

    end

```

```

end

```

```

end

```

```

%% Funktionen diskretisierter Tiefpass

```

```

function G = lpf_pt1_discrete(fc, Ts)
    Om = 2*pi*fc*Ts;           % cut off frequency
    k  = Om/(1+Om);           % Steplenght
    G  = tf(k, [1 -(1-k)], Ts); %Transferfunction

```

```

end

```

```

function G = lpf_discrete(type, fc, Ts)
    if fc == 0 %in case LPF is not activatet
        G = tf(1,1,Ts);
        return;
    end
    switch type
        case 0 % PT1
            G = lpf_pt1_discrete(fc, Ts);

        case 2 % PT2 = PT1 × PT1, mit Cutoff-Correction
            c = 1.553773974;
            H1 = lpf_pt1_discrete(fc*c, Ts);
            G  = H1*H1;

        case 3 % PT3 = PT1 × PT1 × PT1, mit Cutoff-Correction
            c = 1.961459177;
            H1 = lpf_pt1_discrete(fc*c, Ts);
            G  = H1*H1*H1;

        case 1 % BIQUAD: ohne Q keine eindeutige Spec; Placeholder = PT1
            warning('BIQUAD ohne Q: ersetze provisorisch durch PT1.');
```

```

            G = lpf_pt1_discrete(fc, Ts);

        otherwise
            error('Unbekannter Filtertyp');
    end
end
end

```

```

%% Gyro-Lowpassfilter

```

```

%Calculation LPF1

```

```

%Continuous Frequency response

```

```

omega1=2*pi*para_new.gyro_lowpass_hz;

```

```

G_LPF1 = Tiefpassfilter(para_new.gyro_soft_type, omega1, s);

```

```

%Discrete time Frequency response

```

```
G_LPF1_dis = lpf_discrete(para_new.gyro_soft_type, para_new.gyro_lowpass_hz, Ts);
```

```
%Calculation LPF2
```

```
%Continuous Frequency response
```

```
omega2 = 2*pi*para_new.gyro_lowpass2_hz;
```

```
G_LPF2 = Tiefpassfilter(para_new.gyro_soft2_type, omega2, s);
```

```
%Discrete time Frequency response
```

```
G_LPF2_dis = lpf_discrete(para_new.gyro_soft2_type, para_new.gyro_lowpass2_hz, Ts);
```

```
%Calculation LPF1 D-share
```

```
%Continuous Frequency response
```

```
omegad1 = 2*pi*para_new.dterm_lpf_hz;
```

```
G_LPFD1 = Tiefpassfilter(para_new.dterm_filter_type, omegad1, s);
```

```
%Discrete time Frequency response
```

```
G_LPFD1_dis = lpf_discrete(para_new.dterm_filter_type, para_new.dterm_lpf_hz, Ts);
```

```
%Calculation LPF2 D-share
```

```
%Continuous Frequency response
```

```
omegad2 = 2*pi*para_new.dterm_lpf2_hz;
```

```
G_LPFD2 = Tiefpassfilter(para_new.dterm_filter2_type, omegad2, s);
```

```
%Discrete time Frequency response
```

```
G_LPFD2_dis = lpf_discrete(para_new.dterm_filter2_type, para_new.dterm_lpf2_hz, Ts);
```

```
figure(1)
```

```
bode(G_LPF2,G_LPF2_dis, opt);grid on;
```

```
legend('Continuous', 'Discrete','Location', 'southwest');
```

```
title('Lowpassfilter 2');
```

```
figure(2)
```

```
bode(G_LPFD2,G_LPFD2_dis, opt);grid on;
```

```
legend('Continuous', 'Discrete','Location', 'southwest');
```

```
title('Lowpassfilter D-Term 2');
```

```
%% Funktion für Notch-Filter
```

```
function G_notch = Notch(f0, fcut, s)
```

```
    if f0 ~= 0
```

```
        f2 = f0^2 / fcut;
```

```
        Q = f0 / (f2-fcut); % Calculation with cutoff frequency
```

```
        omega = 2*pi*f0;
```

```
        G_notch = (s^2 + omega^2) / (s^2 + (omega/Q)*s + omega^2);
```

```
    else
```

```
        G_notch = tf(1);
```

```
    end
```

```
end
```

```
%% Discretization of Notch filter
```

```
function G_notch_dis = Notch_dis(f0, fcut, Ts, z)
```

```
    if f0 ~= 0
```

```
        f2 = f0^2 / fcut;
```

```
        Q = f0 / (f2-fcut); % Calculation with cutoff frequency
```

```
        omega = 2*pi*f0*Ts;
```

```
        alpha = (1 - sin(omega)) / (2*Q);
```

```
        G_notch_dis = (1 - 2*cos(omega)*z^-1 + z^-2) ...
```

```

        / ((1 + alpha) - 2*cos(omega)*z^-1 + (1 - alpha)*z^-2);
    else
        G_notch_dis = tf(1);
    end
end

%% Notch Filter berechnung

%Berechnung Notchfilter 1
%Continuous Frequency response
G_Notch1 = Notch(para_new.gyro_notch_hz(1), para_new.gyro_notch_cutoff(1), s);
%Discrete time Frequency response
G_Notch1_dis = Notch_dis(para_new.gyro_notch_hz(1), para_new.gyro_notch_cutoff(1), ✓
Ts, z);

%Berechnung Notchfilter 2
%Continuous Frequency response
G_Notch2 = Notch(para_new.gyro_notch_hz(2), para_new.gyro_notch_cutoff(2), s);
%Discrete time Frequency response
G_Notch2_dis = Notch_dis(para_new.gyro_notch_hz(2), para_new.gyro_notch_cutoff(2), ✓
Ts, z);

%Berechnung Notchfilter D-Anteil
%Continuous Frequency response
G_NotchD = Notch(para_new.dterm_notch_hz, para_new.dterm_notch_cutoff, s);
%Discrete time Frequency response
G_NotchD_dis = Notch_dis(para_new.dterm_notch_hz, para_new.dterm_notch_cutoff, Ts, ✓
z);

figure(3)
bode(G_Notch2, G_Notch2_dis,opt); grid on;
legend('Continuous', 'Discrete','Location', 'southwest');
title('Notchfilter 2');

figure(4)
bode(G_NotchD, G_NotchD_dis, opt); grid on;
legend('Continuous', 'Discrete','Location', 'southwest');
title('Notchfilter D-Term 2');

%% Get old PID values

% PID parameters
pid_axis = {'rollPID', 'pitchPID', 'yawPID'};
if (length(para.(pid_axis{ind_ax})) == 5)
    if (para.(pid_axis{ind_ax})(3) ~= para.(pid_axis{ind_ax})(4) && ...
        para.(pid_axis{ind_ax})(4) ~= 0)
        warning([pid_axis{ind_ax}, ' different D gains']);
    end
    % Remove dynamic D-Term
    para.(pid_axis{ind_ax}) = para.(pid_axis{ind_ax})([1 2 3 5]);
end
if para.(pid_axis{ind_ax})(4) ~= 0
    warning([pid_axis{ind_ax}, ' FF is not zero']);
end

```

```

end
% Insert 0 for FF
PID = para.(pid_axis{ind_ax}) .* [get_pid_scale(ind_ax), 0];

%% PID Vektor

pid_scale = [get_pid_scale(ind_ax), 1];

PID_new(1) = P_new * pid_scale(1); % Proportionalanteil

%KI von KP abhängig, daher rückrechnung nötig
fI = PID(2) / (2 * pi * PID(1)); % alte Integralfrequenz aus altem PID
fI_new = fI * I_ratio_new; % gewünschte skalierte Integralfrequenz

PID_new(2) = 2 * pi * PID_new(1) * fI_new; % Integralanteil
%Differentialanteil unabhängig
PID_new(3) = D_new * pid_scale(3); % Differentialanteil
PID_new(4) = 0;

%% Ausgabe PI und D Regler

C_PI = PID_new(1) + PID_new(2)/s; %PI Regler ohne Filter
Kp = tf(PID_new(1)); %Für Simulink
C_PI_LFP_Notch = C_PI * G_LPF1 * G_LPF2 * G_Notch1 * G_Notch2; %PI Regler mit Filter
Filter

C_D = PID_new(3)*s; %D Regler ohne Filter
C_D_LFP = C_D*G_LPFD1*G_LPFD2;
C_D_LFP_Notch = C_D* G_LPFD1*G_LPFD2*G_NotchD; %D Regler mit Filter

%% PID Vektor Diskretisiert

% Calculation of discretization of PI Part of the Controller

C_I_dis = (PID_new(2)*Ts) / (1 - z^-1); % Calculation I Part
C_PI_dis = PID_new(1) + C_I_dis; % Combination P and I Part of Controller
C_PI_LFP_Notch_dis = C_PI_dis * G_LPF1_dis * G_LPF2_dis; %PI Controller with LPF

%Calculation of D Part of the Controller
C_D_dis = (PID_new(3)*(1 - z^-1))/Ts; %Calculation D Part
C_D_LFP_dis = C_D_dis*G_LPFD1_dis*G_LPFD2_dis; %D Part with LPF

figure(5)
bode(C_PI, C_PI_dis, C_D_LFP, C_D_LFP_dis, opt);grid on;
legend('Continuous PI', 'Discrete PI', 'Continuous D', 'Discrete D', ...
'Location', 'southeast');
title('PI and D with LPF Controller');

%% Simulation Strecke P

switch ind_ax
case 1 %Roll
%Relevante Frequenzen

```

```

w2 = 13*2*pi();
w1 = 16*2*pi();
wt = 60*2*pi();

```

```

%Übertragungsfunktionen abgeschätzt

```

```

G1 = 1 / (s/w1);
G2 = 1 / (1 + (s/w2));
Gt = exp(-s * (1/wt));      %Totzeit
P_ges = G1*G2*Gt;          %Gesamtübertragungsfunktion

```

```

case 2 %Pitch

```

```

%Relevante Frequenzen

```

```

w2 = 10*2*pi();
w1 = 13*2*pi();
wt = 60*2*pi();

```

```

%Übertragungsfunktionen abgeschätzt

```

```

G1 = 1 / (s/w1);
G2 = 1 / (1 + (s/w2));
Gt = exp(-s * (1/wt));      %Totzeit
P_ges = G1*G2*Gt;          %Gesamtübertragungsfunktion

```

```

case 3 %Yaw

```

```

%Relevante Frequenzen

```

```

w2 = 60*2*pi();
w1 = 10*2*pi();
wt = 60*2*pi();

```

```

%Übertragungsfunktionen abgeschätzt

```

```

G1 = 1 / (s/w1);
G2 = 1 / (1 + (s/w2));
Gt = exp(-s * (1/wt));      %Totzeit
P_ges = G1*G2*Gt;          %Gesamtübertragungsfunktion

```

```

end

```

```

P_ges_d = c2d(P_ges, Ts, 'thiran'); % Discretisation of Plant

```

```

%% Real Plant

```

```

addpath ../Simulations

```

```

load ('real_plant_apex.mat'); %Get Data from bf_controller_tuning

```

```

figure(6)

```

```

bode(P/Gf_ana , P_ges_d, opt);

```

```

legend('Measured','Simulated','Location','southeast');

```

```

title('Plant, Measured and Simulated');

```

```

%% Simulated Transferfunction

```

```

%Continuous Frequency response

```

```

% Inner Open loop

```

```

G_oli = P_ges * G_Notch1 * G_Notch2 * G_LPF1 * G_LPF2;
G_cli = G_oli / (1+G_oli*G_NotchD*G_LPFD1*G_LPFD2);

```

```

%Outer Loop

```

```

G_olo = G_cli * C_PI;
G_olc = G_olo / (1+ G_olo);

```

```

% Discret Frequency response

```

```
% Inner Open loop
```

```
G_oli_dis = P_ges_d * G_Notch1_dis * G_Notch2_dis * G_LPF1_dis * G_LPF2_dis;  
G_cli_dis = G_oli_dis / (1+G_oli_dis*G_NotchD_dis*G_LPFD1_dis*G_LPFD2_dis);
```

```
%Outer Loop
```

```
G_olo_dis = G_cli_dis * C_PI_dis;  
G_olc_dis = G_olo_dis / (1+ G_olo_dis);
```

```
%% Step response Drone
```

```
figure(8)  
plot(step_time, step_resp_pl), grid on  
xlim([0 , 0.5])
```

```
%Step does not work rigth at the moment
```

```
figure(9)  
step(G_olc_dis);
```